

Databasesystemer 2024

Deleksamen 1

Gruppe 48

Andreas Isaksen

Robert Johannessen

Ivar Kvalheim

Innholdsfortegnelse

Introduksjon	3
Kravspesifikasjon	3
Normalisering	5
ER-modellering	10
Database implementasjon	11
Dummy data	17
SQL-spørringer.....	25
Skjermdump fra view spørringer	29
Konklusjon.....	32
Litteraturliste	33

Introduksjon

I denne rapporten blir det gjennomgått hele prosessen med å utvikle en database til en kunde som er ute etter et komplett system for trening på profesjonelt nivå.

Det vil si, denne databasen skal kunne brukes til å lage treningsplaner, som kan justeres etter hver utøver. Denne skal bestå av diverse øvelser som trenerne setter opp. Treningsplanene skal da kunne bli brukt av utøverne, og ha funksjoner som gjør at utøverne kan logge hver økt, som igjen kan bli brukt til å se progresjonen til hver enkelt utøver.

Dette betyr at hver enkelt person må ha tilpasset hva de kan gjøre, bruke og se i databasen.

Ambisjonene når databasen ble konstruert, var at systemet skulle være så fleksibelt som mulig, slik at det var gode muligheter til å justere hver type trening og at man har mulighet til å vise forskjellig data til forskjellige anledninger.

Kravspesifikasjon

- Kundens ønsker
 - Kunde ønsker et system som skal benyttes av trenere og utøvere.
 - Her vil det da være behov for to forskjellige adgangs nivåer. Et som gir kunden tilgang til sine egne data og noe skrive rettigheter, og et for ansatte som gir tilgang til all data, samt utvidede skrive rettigheter. Dette kan løses ved å opprette forskjellige 'views' for kunder og ansatte.
 - (tenkt insert script) Trener skal ha tilgang til å lage individuelle treningsplaner.
 - Dette kan løses ved å gi ansatte editings rettigheter i en 'treningsplan entitet' med kobling til de nødvendige data.
 - Treningsplan skal inneholde flere forskjellige økter.
 - Dette har vi løst ved å lage en koblingstabell med navn 'øktTyper' som kobles mot de forskjellige øktene.
 - Økter skal bestå av følgende kategorier:
 - Kondisjonsøkt (Kortintervall (høy intensitet), Alternativ kortintervall, Langintervall (moderat intensitet), Hurtigkjøring i kupert terreng) (SATS, 2023)
 - Styrkeøkt (Knedominant øvelse, Hoftedominant øvelse, Vertikal trekkøvelse, Horisontal trekkøvelse, Vertikal pressøvelse, Horisontal pressøvelse,) (Holmemo, 2019)
 - Konkurransen (Oppdatert informasjon om hvilken konkurranse det er og hva slags type.)

- Trener skal ha mulighet for å gjenbruke tidligere deklarererte øvelser.
 - Dette har vi løst ved å ikke knytte treningsplaner til utøvere direkte til treningsplaner, men at utøver kan knyttes til en treningsplan gjennom hver enkelt utført økt.
- Utøver bør ha tilgang til en forklaring på gjennomgang av øvelser.
 - Dette kan løses ved å lage en 'view' funksjon for bruker som gir tilgang til enkelte attributter i øvelse entiteten.
- Utøver må ha tilgang til sine respektive treningsplaner og utførte økter, slik som pulssoner i løpet av kondisjonsøkter(Holmemo, 2019).
 - Dette kan løses ved å opprette 'view' funksjoner for utøver som gir en lesertilgang på utøvers data i entitetene treningsplaner og utførte økter. Disse 'view' funksjonen må restrikeres slik at utøver kun får tilgang til sine egne respektive data og ikke andre utøver.
- Styrkeøkter skal kunne føre inn vekt nivå som er benyttet på forskjellige økter.
 - Dette kan løses ved å gi utøver skriverettigheter på de relevante attributtene i entiteten økter, som er knyttet mot rader som har en relasjon til styrkeøkter via en entitet som holder på økt kategorier.
- Utøver skal ha mulighet til å legge inn kommentarer på sine økter(treningsdagbok).
 - Dette kan løses ved å gi utøver skriverettigheter i en kommentar attributt i utøvers utførte øvelser.
- Treningsdagbok skal være tilgjengelig for trener og utøver.
 - Dette kan løses ved å opprette to separate 'view' funksjoner for utøver og trener. Utøver vil ha tilgang til sine egne data i treningsdagbok, og trener vil ha tilgang til all data i entiteten.
- Det skal foreligge en historikk på utøver for å kunne se progresjon.
 - Dette kan løses ved å lage en 'view' funksjon som henter ut data på alle utførte øvelser tilhørende den omhandlende utøver.
- Løsningsforslag
 - For å innfri kundens overnevnte ønsker starter vi med å sette opp en tenkt oversikt med entiteter som skal brukes til å videre opprette en databaseløsning. I starten så ser vi for oss følgende entiteter før vi knytter attributter til disse:
 - Utøver – inneholder data om utøver
 - Trener – Inneholder data om trener
 - Økt typer – Inneholder beskrivelse av økter, med forklaring om utførelse.
 - Treningsplan – Inneholder data om treningsplan.
 - Økter – Inneholder data om utførte økter, og brukes til historikk på utøver.

Normalisering

Start:

Her er det vi fant mest logisk å ta med av kravspesifikasjoner fra kunden vår:

Utøver – Trener – Treningsplan - Økt-typer – Økter - Vanskelighet/nivå - Historikk - Kontakt info
 Vekt - Varighet - Plassering i konkurranse – Tidspunkt – Pulssone - Beskrivelse av øvelser –
 Kommentarer

Finne entiteter og tilhørende attributter som skal gi oss et grunnlag for hvordan databasen kan se ut:

-Utøver (Id, Fornavn, Etternavn, Adresse, Tlf)

-Trener (Id, Fornavn, Etternavn, Adresse, Tlf)

-Økt typer (Styrke, Kondisjon, Konkurranse, Nivå)

-Treningsplan (Økt-type, Nivå)

-Økter (Løpenummer, Id, Dato, Plan-id, Økt-id, Vekt, varighet, Pulssone, Plassering, Kommentar)

Når vi startet oppretting av en oversikt på nødvendige attributter, så fant vi det naturlig å dele det opp i respektive entiteter. Disse har vi videre brukt til å gjøre en videre normaliseringsprosess.

Utøver	Trener	Økt typer	Treningsplan	Økter
<u>Kunde_id</u> Fornavn Etternavn Adresse Tlf	<u>Ansatt_id</u> Fornavn Etternavn Adresse Tlf	<u>Økt_id</u> Økt_type Beskrivelse Øvelse Konkurranse	<u>Plan_id</u> p_id* Ansatt_id* Økt_type* Nivå	<u>Løpenummer</u> p_id* Dato Pulssoner Vekt Varighet Plassering Økt_id* Plan_id* Kommentar

1NF - 2NF – 3NF (utøver, ansatt):

Med tanke på hvordan vi startet med entitet og attributt strukturen vår, kunne vi tidlig slå fast at det ikke forekom noen atomære verdier i disse entitetene og separat var det heller ingen attributter som hadde like navn beskrivelser. Dermed anså vi disse til å ha innfridd kravene for første normalform. Videre når vi startet å se etter potensielle kandidatnøkler, så vi at det forekom noen partielle avhengigheter mellom attributtene; 'id', 'tlf' og potensielt 'adresse'. På grunnlag av dette vedtok vi at disse entitetene burde brytes opp i flere entiteter for å utfylle kravene for andre normalform. Entitetene 'utøver' og 'ansatt' ble dermed brutt ned til følgende entiteter:

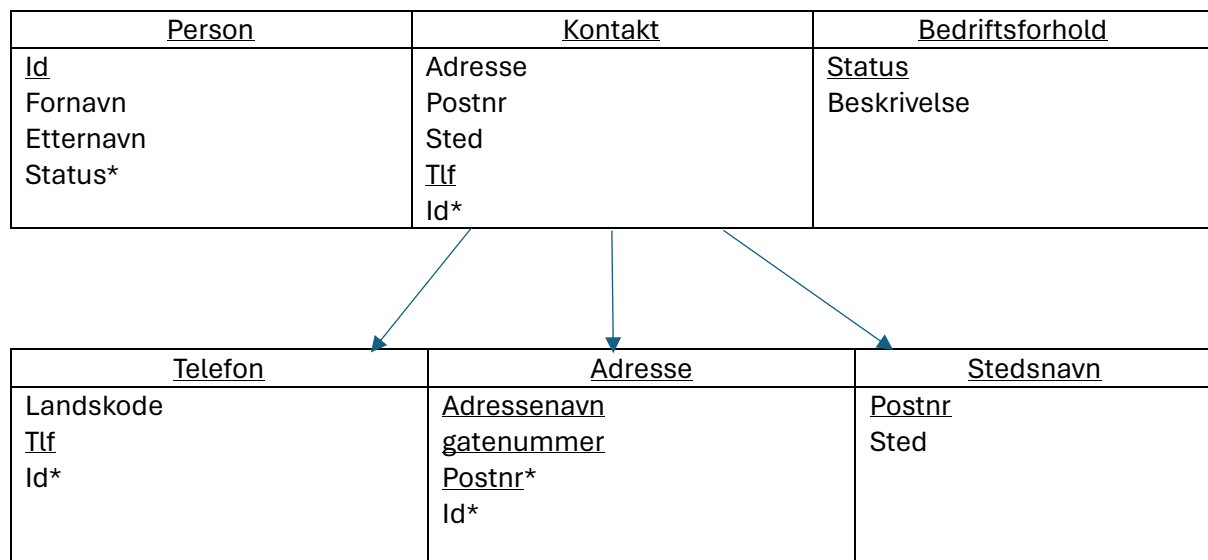
- **Person** (id, fornavn, etternavn, status*) – Denne entiteten holder nå på navn til trenere og utøvere samlet.
- **Kontakt** (adresse, postnr, sted, tlf, id*) – Denne entiteten holder på kontakt data for person objekter.
- **Bedriftsforhold** (status, beskrivelse) – Denne entiteten holder på en status verdi som definerer om objektet er trener eller utøver.

Med denne løsningen var det ikke lenger noen partielle avhengigheter i entitetene og vi hadde utfyllt kravet for andre normalform.

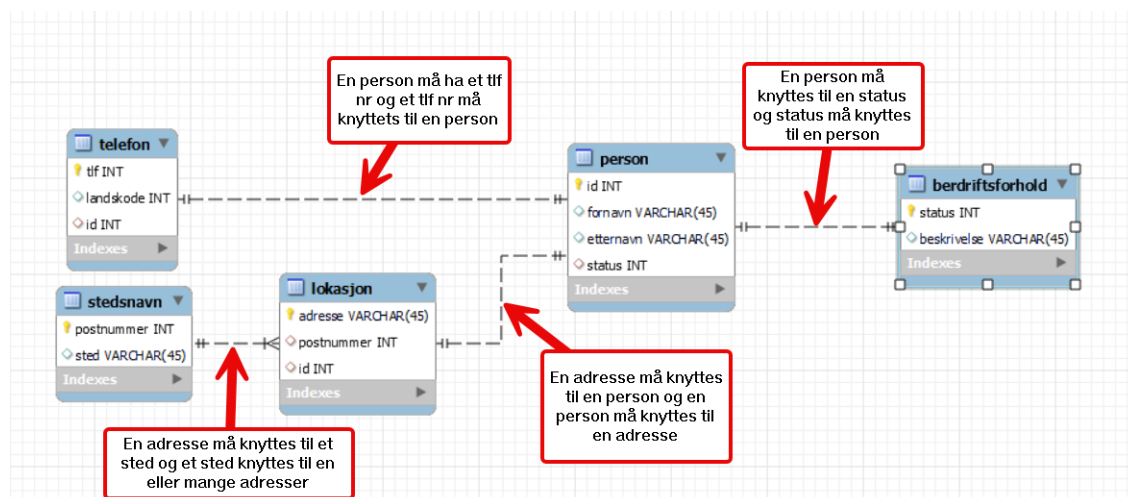
For tredje normalform kunne vi se at entitetene 'person' og 'bedriftsforhold' allerede utfylte disse, men 'kontakt' entiteten hadde noen potensielle transitive avhengigheter siden attributtene 'tlf', 'adressenavn', 'postnr' og sted kunne være potensielle kandidatnøkler. Dermed brøt vi 'kontakt' entiteten ned i til tre nye entiteter:

- **Telefon** (landskode, tlf, id*) – Denne entiteten holder på telefon nummer til person objekter.
- **Adresse** (adressenavn, gatenummer, postnr*, id*) – Denne entiteten holder på adressenavn tilhørende person objekter.
- **Stedsnavn** (postnr, sted) – Denne entiteten holder på stedsnavn tilhørende adresse objekter.

Med denne løsningen skal det ikke forekomme noen transitive avhengigheter og vi kan anse at denne dele av databasestrukturen nå utfyller tredje normalform. Det er også verdt å nevne at denne løsningen gir heller ingen ikke-trivielle funksjonelle avhengigheter, som vil si at den også innfrir kravene for BCNF.



Her er et utklipp fra ER diagrammet som illustrerer hvordan persondata blir strukturert i databasen.



1NF - 2NF – 3NF (Økt typer, treningsplan, økter):

I prosessen med å normalisere «Økt typer», «Treningsplan» og «Økter», fant vi fort ut at vi måtte dele opp entitetene litt og legge til noen ekstra attributter. Vi var ute etter en så fleksibel og dynamisk database som mulig der vi kunne justere hvert aspekt av treningsplanen.

Økt typer	Treningsplan	Økter
<u>Økt_id</u> Økt_type Beskrivelse Øvelse Konkurranse	<u>Plan_id</u> p_id* Ansatt_id* Økt_type* Nivå	<u>Løpenummer</u> P_id* Dato Pulssoner Vekt Varighet Plassering Økt_id* Plan_id* Kommentar

Dette ble blant annet løst ved å bryte opp «Økt typer» slik at vi fikk en tabellstruktur for hver av de tre økt typene.

- **øktTyper** (id, kategori) - I denne entiteten kan man velge mellom de 3 hovedtypene under:

- **konkurranse** (katagori, event, beskrivelse) - Type konkurranse og konkurranse navn.
- **kondisjonsOvelser** (kategori, ovelse, beskrivelse) - Alle øvelsene.
- **styrkeTrening** (kategori, treningstyper) - Trening oppdelt i kroppsdeler. (undertabell for styrkeØvelser.)
- **styrkeOvelser** (ovelse, beskrivelse, treningstyper) - De spesifikke øvelsene man har i styrke.
- **øktBeskrivelse** – (b_id, beskrivelse) - Beskrivelse av de 3 forskjellige øvelsene.

Vi støtet på et problem med «**beskrivelse**» under noen spørringer. Vi hadde denne attributten i hver enkelt tabell først, men valgte å flytte den over i en egen tabell, for smidigere håndtering av databasen.

Nå har vi også fått gode koblinger mellom tabellene og fått bort alle avhengigheter.

Videre så ble det gjort litt justeringer på «**Treningsplan**».

Vi måtte legge nivå med beskrivelse i egen tabell for å slippe noen avhengigheter i treningsplan.

- **Nivå** (n-id, beskrivelse) - Her kan man justere hvor mange repetisjoner som skal gjøres i øvelsene, slik at trenere kan justere nivået på treningsplanen og økt typene.

Vi la til løpenummer som vi fant ut at var beste løsning for en kandidatnøkkel, som senere ble Primær nøkkel.

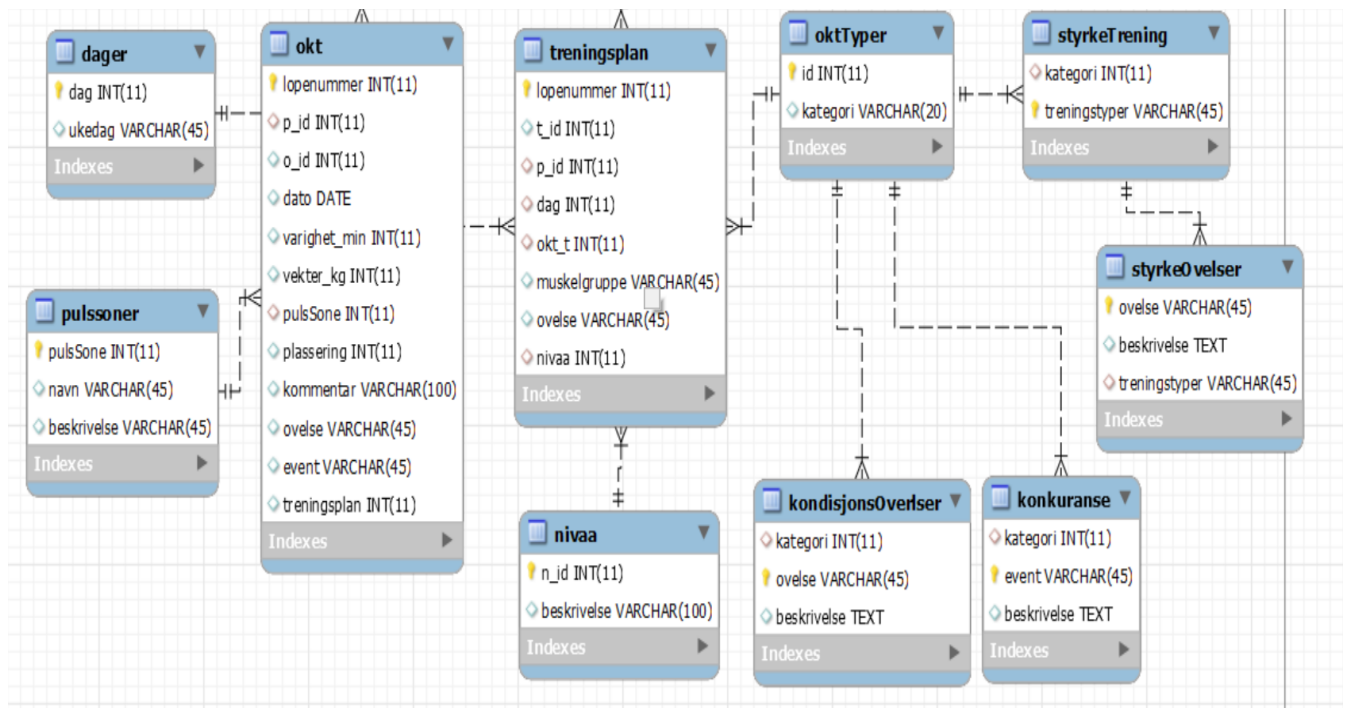
Vi har med en id også, for å kunne vise til og gjenbruke treningsplanene.

Etter det fikk vi løst både partielle og transitive avhengigheter relativt greit.

- **Treningsplan** (lopenummer, t_id, p_id, dag, økt_id, muskelhruppe, ovelse, nivaa) – En entitet som ble større etter start, men som nå inneholder en nøyaktig og dynamisk måte å sette opp forskjellige treningsplaner, ved å bruke data fra hele databasen.
-

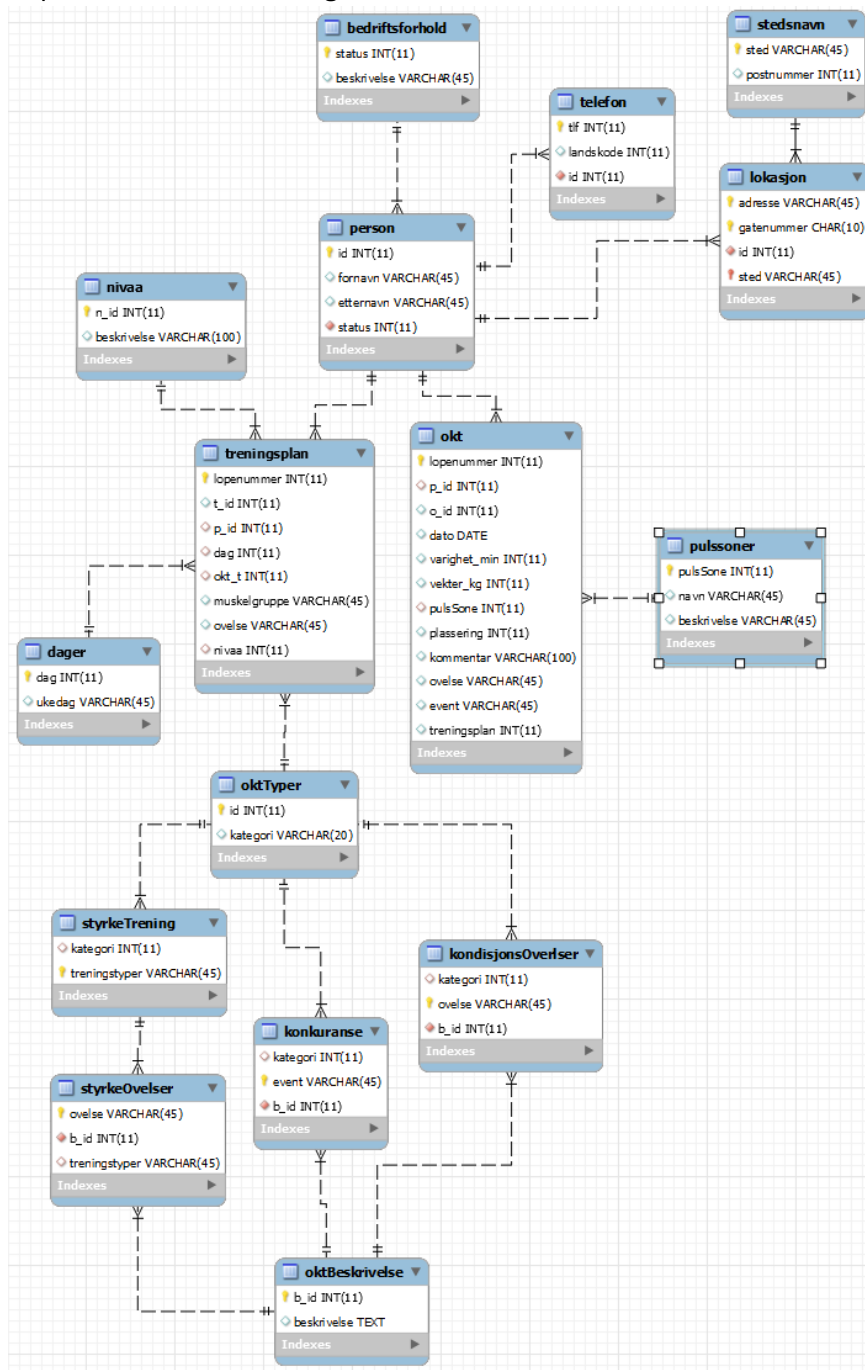
Økter ble «**økt**» og skal være omtrent som en treningsdagbok. Denne henter mye fra andre tabeller og var faktisk helt ok å sette opp med hensyn til avhengigheter og normalisering.

- **økt** (lopenummmmer, p_id, o_id, dato, varighet_min, vektor_kg, pulsSone, plassering, kommentar, ovelse, event, treningsplan) – En entitet som inneholder all informasjon om resultater fra hver enkelt økt til utøverne som de kan logge. Dette igjen gjør det enkelt å produsere views om progresjonen.



ER-modellering

- Dette er slutt modellen på vår ER diagram som vi videre brukte som plantegning for å implementere den endelige database strukturen



Database implementasjon

For å implementere databasen brukte vi ER-diagrammet MySQL workshop, som vi videre brukte 'forward engineeer' funksjonen til å produsere SQL kode til å bygge tabell og koblingsstruktur. Dette er SQL koden vi brukte til å bygge database strukturen:

```
-- MySQL Workbench Forward Engineering

SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE,
SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';

-- -----
-- Table `db2024_G48`.`bedriftsforhold`
-- -----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`bedriftsforhold` (
  `status` INT(11) NOT NULL,
  `beskrivelse` VARCHAR(45) NULL DEFAULT NULL,
  PRIMARY KEY (`status`),
  UNIQUE INDEX `status_UNIQUE` (`status` ASC) VISIBLE)
ENGINE = InnoDB;

-- -----
-- Table `db2024_G48`.`dager`
-- -----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`dager` (
  `dag` INT(11) NOT NULL,
  `ukedag` VARCHAR(45) NULL DEFAULT NULL,
  PRIMARY KEY (`dag`))
ENGINE = InnoDB;

-- -----
-- Table `db2024_G48`.`oktBeskrivelse`
-- -----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`oktBeskrivelse` (
  `b_id` INT(11) NOT NULL AUTO_INCREMENT,
  `beskrivelse` TEXT NULL DEFAULT NULL,
  PRIMARY KEY (`b_id`))
ENGINE = InnoDB
AUTO_INCREMENT = 26;

-- -----
-- Table `db2024_G48`.`oktTyper`
-- -----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`oktTyper` (
  `id` INT(11) NOT NULL,
  `kategori` VARCHAR(20) NULL DEFAULT NULL,
  PRIMARY KEY (`id`))
```

```
ENGINE = InnoDB;
```

```
-- Table `db2024_G48`.`kondisjonsOverlser`
```

```
CREATE TABLE IF NOT EXISTS `db2024_G48`.`kondisjonsOverlser` (
  `kategori` INT(11) NULL DEFAULT NULL,
  `ovelse` VARCHAR(45) NOT NULL,
  `b_id` INT(11) NOT NULL,
  PRIMARY KEY (`ovelse`),
  INDEX `fk_kondisjonsOvelser_oktTyper_idx` (`kategori` ASC) VISIBLE,
  INDEX `fk_kondisjonsOvelser_oktBeskrivelse_idx` (`b_id` ASC) VISIBLE,
  CONSTRAINT `fk_kondisjonsOvelser_oktBeskrivelse`
    FOREIGN KEY (`b_id`)
    REFERENCES `db2024_G48`.`oktBeskrivelse` (`b_id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `fk_kondisjonsOvelser_oktTyper`
    FOREIGN KEY (`kategori`)
    REFERENCES `db2024_G48`.`oktTyper` (`id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

```
-- Table `db2024_G48`.`konkuranse`
```

```
CREATE TABLE IF NOT EXISTS `db2024_G48`.`konkuranse` (
  `kategori` INT(11) NULL DEFAULT NULL,
  `event` VARCHAR(45) NOT NULL,
  `b_id` INT(11) NOT NULL,
  PRIMARY KEY (`event`),
  INDEX `fk_konkuranse_oktTyper_idx` (`kategori` ASC) VISIBLE,
  INDEX `fk_konkuranse_oktBeskrivelse_idx` (`b_id` ASC) VISIBLE,
  CONSTRAINT `fk_konkuranse_oktBeskrivelse`
    FOREIGN KEY (`b_id`)
    REFERENCES `db2024_G48`.`oktBeskrivelse` (`b_id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `fk_konkuranse_oktTyper`
    FOREIGN KEY (`kategori`)
    REFERENCES `db2024_G48`.`oktTyper` (`id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

```
-- Table `db2024_G48`.`stedsnavn`
```

```
CREATE TABLE IF NOT EXISTS `db2024_G48`.`stedsnavn` (
  `sted` VARCHAR(45) NOT NULL,
  `postnummer` INT(11) NULL DEFAULT NULL,
  PRIMARY KEY (`sted`))
ENGINE = InnoDB;
```

```

-----
-- Table `db2024_G48`.`person`
-----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`person` (
  `id` INT(11) NOT NULL,
  `fornavn` VARCHAR(45) NULL DEFAULT NULL,
  `etternavn` VARCHAR(45) NULL DEFAULT NULL,
  `status` INT(11) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE INDEX `id_UNIQUE` (`id` ASC) VISIBLE,
  INDEX `fk_person_bedriftsforhold_idx` (`status` ASC) VISIBLE,
  CONSTRAINT `fk_person_bedriftsforhold`
    FOREIGN KEY (`status`)
      REFERENCES `db2024_G48`.`bedriftsforhold` (`status`)
      ON DELETE NO ACTION
      ON UPDATE NO ACTION)
ENGINE = InnoDB;

-----
-- Table `db2024_G48`.`lokasjon`
-----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`lokasjon` (
  `adresse` VARCHAR(45) NOT NULL,
  `gatenummer` CHAR(10) NOT NULL,
  `id` INT(11) NOT NULL,
  `sted` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`adresse`, `gatenummer`, `sted`),
  INDEX `fk_loksjon_person_idx` (`id` ASC) VISIBLE,
  INDEX `fk_lokajson_stedsnavn_idx` (`sted` ASC) VISIBLE,
  CONSTRAINT `fk_lokajson_stedsnavn`
    FOREIGN KEY (`sted`)
      REFERENCES `db2024_G48`.`stedsnavn` (`sted`)
      ON DELETE NO ACTION
      ON UPDATE NO ACTION,
  CONSTRAINT `fk_loksjon_person`
    FOREIGN KEY (`id`)
      REFERENCES `db2024_G48`.`person` (`id`)
      ON DELETE NO ACTION
      ON UPDATE NO ACTION)
ENGINE = InnoDB;

-----
-- Table `db2024_G48`.`nivaa`
-----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`nivaa` (
  `n_id` INT(11) NOT NULL,
  `beskrivelse` VARCHAR(100) NULL DEFAULT NULL,
  PRIMARY KEY (`n_id`))
ENGINE = InnoDB;

-----
-- Table `db2024_G48`.`pulssoner`
-----

```

```
-----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`pulssoner` (
  `pulsSone` INT(11) NOT NULL,
  `navn` VARCHAR(45) NULL DEFAULT NULL,
  `beskrivelse` VARCHAR(45) NULL DEFAULT NULL,
  PRIMARY KEY (`pulsSone`))
ENGINE = InnoDB;
```

```
-----
-- Table `db2024_G48`.`okt`
-----
```

```
CREATE TABLE IF NOT EXISTS `db2024_G48`.`okt` (
  `lopenummer` INT(11) NOT NULL AUTO_INCREMENT,
  `p_id` INT(11) NULL DEFAULT NULL,
  `o_id` INT(11) NULL DEFAULT NULL,
  `dato` DATE NULL DEFAULT NULL,
  `varighet_min` INT(11) NULL DEFAULT NULL,
  `vekker_kg` INT(11) NULL DEFAULT NULL,
  `pulsSone` INT(11) NULL DEFAULT NULL,
  `plassering` INT(11) NULL DEFAULT NULL,
  `kommentar` VARCHAR(100) NULL DEFAULT NULL,
  `ovelse` VARCHAR(45) NULL DEFAULT NULL,
  `event` VARCHAR(45) NULL DEFAULT NULL,
  `treningsplan` INT(11) NULL DEFAULT NULL,
  PRIMARY KEY (`lopenummer`),
  INDEX `fk_okt_person_idx` (`p_id` ASC) VISIBLE,
  INDEX `fk_okt_pulssoner` (`pulsSone` ASC) VISIBLE,
  CONSTRAINT `fk_okt_person`
    FOREIGN KEY (`p_id`)
      REFERENCES `db2024_G48`.`person` (`id`)
      ON DELETE NO ACTION
      ON UPDATE NO ACTION,
  CONSTRAINT `fk_okt_pulssoner`
    FOREIGN KEY (`pulsSone`)
      REFERENCES `db2024_G48`.`pulssoner` (`pulsSone`))
ENGINE = InnoDB
AUTO_INCREMENT = 13;
```

```
-----
-- Table `db2024_G48`.`styrkeTrening`
-----
```

```
CREATE TABLE IF NOT EXISTS `db2024_G48`.`styrkeTrening` (
  `kategori` INT(11) NULL DEFAULT NULL,
  `treningstyper` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`treningstyper`),
  INDEX `fk_styrketrening_oktTyper_idx` (`kategori` ASC) VISIBLE,
  CONSTRAINT `fk_styrketrening_oktTyper`
    FOREIGN KEY (`kategori`)
      REFERENCES `db2024_G48`.`oktTyper` (`id`)
      ON DELETE NO ACTION
      ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

```
-- Table `db2024_G48`.`styrkeOvelser`
-----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`styrkeOvelser` (
  `ovelse` VARCHAR(45) NOT NULL,
  `b_id` INT(11) NOT NULL,
  `treningstyper` VARCHAR(45) NULL DEFAULT NULL,
  PRIMARY KEY (`ovelse`),
  INDEX `fk_styrkeTrening_styrkeOvelser_idx` (`treningstyper` ASC) VISIBLE,
  INDEX `fk_styrkeOvelser_oktBeskrivelse_idx` (`b_id` ASC) VISIBLE,
  CONSTRAINT `fk_styrkeOvelser_oktBeskrivelse`
    FOREIGN KEY (`b_id`)
      REFERENCES `db2024_G48`.`oktBeskrivelse` (`b_id`)
      ON DELETE NO ACTION
      ON UPDATE NO ACTION,
  CONSTRAINT `fk_styrkeTrening_styrkeOvelser`
    FOREIGN KEY (`treningstyper`)
      REFERENCES `db2024_G48`.`styrkeTrening` (`treningstyper`)
      ON DELETE NO ACTION
      ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

```
-- Table `db2024_G48`.`telefon`
-----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`telefon` (
  `tlf` INT(11) NOT NULL,
  `landskode` INT(11) NULL DEFAULT NULL,
  `id` INT(11) NOT NULL,
  PRIMARY KEY (`tlf`),
  UNIQUE INDEX `tlf_UNIQUE` (`tlf` ASC) VISIBLE,
  INDEX `fk_person_telefon_idx` (`id` ASC) VISIBLE,
  CONSTRAINT `fk_person_telefon`
    FOREIGN KEY (`id`)
      REFERENCES `db2024_G48`.`person` (`id`)
      ON DELETE NO ACTION
      ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

```
-- Table `db2024_G48`.`treningsplan`
-----
CREATE TABLE IF NOT EXISTS `db2024_G48`.`treningsplan` (
  `lopenummer` INT(11) NOT NULL AUTO_INCREMENT,
  `t_id` INT(11) NULL DEFAULT NULL,
  `p_id` INT(11) NULL DEFAULT NULL,
  `dag` INT(11) NULL DEFAULT NULL,
  `okt_t` INT(11) NULL DEFAULT NULL,
  `muskelgruppe` VARCHAR(45) NULL DEFAULT NULL,
  `ovelse` VARCHAR(45) NULL DEFAULT NULL,
  `nivaa` INT(11) NULL DEFAULT NULL,
  PRIMARY KEY (`lopenummer`),
  INDEX `fk_treningsplan_person_idx` (`p_id` ASC) VISIBLE,
  INDEX `fk_treningsplan_dager_idx` (`dag` ASC) VISIBLE,
  INDEX `fk_treningsplan_oktTyper_idx` (`okt_t` ASC) VISIBLE,
  INDEX `bhjbsi_idx` (`nivaa` ASC) VISIBLE,
```

```
CONSTRAINT `fk_treningsplan_dager`  
  FOREIGN KEY (`dag`)  
    REFERENCES `db2024_G48`.`dager` (`dag`)  
  ON DELETE NO ACTION  
  ON UPDATE NO ACTION,  
CONSTRAINT `fk_treningsplan_nivaa`  
  FOREIGN KEY (`nivaa`)  
    REFERENCES `db2024_G48`.`nivaa` (`n_id`)  
  ON DELETE NO ACTION  
  ON UPDATE NO ACTION,  
CONSTRAINT `fk_treningsplan_oktTyper`  
  FOREIGN KEY (`okt_t`)  
    REFERENCES `db2024_G48`.`oktTyper` (`id`)  
  ON DELETE NO ACTION  
  ON UPDATE NO ACTION,  
CONSTRAINT `fk_treningsplan_person`  
  FOREIGN KEY (`p_id`)  
    REFERENCES `db2024_G48`.`person` (`id`)  
  ON DELETE NO ACTION  
  ON UPDATE NO ACTION)  
ENGINE = InnoDB  
AUTO_INCREMENT = 31;  
  
SET SQL_MODE=@OLD_SQL_MODE;  
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;  
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
```

Dummy data

Ved generering av større mengder med dummy data har vi tatt i bruk chatGPT(OpenAI, 2023). Større mengder data omhandler de data som finnes i person data entiteter, med unntak av 'bedriftsforhold' entiteten. All annen data i databasen er manuelt lagt inn med 'INSERT INTO' spørringer.

Ved innsetting av dummy data til databasesystemet ble import wizard, som er en innebygd funksjon for importering av data i MySQL Workbench til tabeller, benyttet til å importere data til følgende tabeller:

- bedriftsforhold
- dager
- nivaa
- okt
- oktTyper
- pulssoner
- styrkeTrening
- treningsplan

Til tabellene:

- stedsnavn
- telefon
- lokasjon
- styrkeOvelser
- kondisjonsOverlser
- konkuranse
- oktbeskrivelse

Ble dummy data satt inn i tabellene ved å bruke sql-scripts. Alt av dummy data som er i dette dokumentet, er plassert i den rekkefølgen som den skal settes inn i ved replikasjon av denne databasen.

```
1 -- -----
2 -- Dummy data bedriftsforhold
3 -- -----
4 INSERT INTO bedriftsforhold(status, beskrivelse)
5 VALUES(0, 'trener'),
6 (1, 'utover');
```

```
1 -- -----
2 -- Dummy data stedsnavn
3 -- -----
4 INSERT INTO stedsnavn(sted, postnummer)
5 VALUES ('CityA', 1234),
6 ('CityAA', 8765),
7 ('CityAB', 7654),
8 ('CityAC', 5432),
9 ('CityAD', 4321),
10 ('CityAE', 3210),
11 ('CityAF', 2109),
12 ('CityAG', 1098),
13 ('CityAH', 987),
14 ('CityAI', 9876),
```

```

15 ('CityAJ', 8765),
16 ('CityAK', 7654),
17 ('CityAL', 5432),
18 ('CityAM', 4321),
19 ('CityAN', 3210),
20 ('CityAO', 2109),
21 ('CityAP', 1098),
22 ('CityAQ', 987),
23 ('CityAR', 9876),
24 ('CityAS', 8765),
25 ('CityAT', 7654),
26 ('CityAU', 5432),
27 ('CityAV', 4321),
28 ('CityAW', 3210),
29 ('CityAX', 2109),
30 ('CityAY', 1098),
31 ('CityAZ', 987),
32 ('CityB', 5678),
33 ('CityBA', 9876),
34 ('CityBB', 8765),
35 ('CityBC', 7654),
36 ('CityBD', 5432),
37 ('CityBE', 4321),
38 ('CityBF', 3210),
39 ('CityBG', 2109),
40 ('CityBH', 1098),
41 ('CityBI', 987),
42 ('CityBJ', 9876),
43 ('CityBK', 8765),
44 ('CityBL', 7654),
45 ('CityBM', 5432),
46 ('CityBN', 4321),
47 ('CityBO', 3210),
48 ('CityBP', 2109),
49 ('CityBQ', 1098),
50 ('CityBR', 987),
51 ('CityBS', 9876),
52 ('CityBT', 8765),
53 ('CityBU', 7654),
54 ('CityBV', 5432),
55 ('CityBW', 4321),
56 ('CityBX', 3210),
57 ('CityBY', 2109),
58 ('CityBZ', 1098),
59 ('CityC', 9101),
60 ('CityCA', 987),
61 ('CityCB', 9876),
62 ('CityCC', 8765),
63 ('CityCD', 7654),
64 ('CityCE', 5432),
65 ('CityCF', 4321),
66 ('CityCG', 3210),
67 ('CityCH', 2109),
68 ('CityCI', 1098),
69 ('CityD', 1121),
70 ('CityE', 3141),
71 ('CityF', 5161),
72 ('CityG', 7181),
73 ('CityH', 9201),
74 ('CityI', 1221),
75 ('CityJ', 3241),
76 ('CityK', 7890),
77 ('CityL', 5432),
78 ('CityM', 2109),
79 ('CityN', 1098),
80 ('CityO', 987),
81 ('CityP', 9876),
82 ('CityQ', 8765),
83 ('CityR', 7654),
84 ('CityS', 6543),
85 ('CityT', 5432),
86 ('CityU', 4321),
87 ('CityV', 3210),
88 ('CityW', 2109),
89 ('CityX', 1098),
90 ('CityY', 987),
91 ('CityZ', 9876);

```

```

1  -- -----
2  -- Dummy data person
3  -- -----
4  INSERT INTO person(id, fornavn,
5  etternavn, status)
6  VALUES(1, 'John', 'Doe', 0),
7  (2, 'Jane', 'Doe', 0),
8  (3, 'Alice', 'Johnson', 0),
9  (4, 'Bob', 'Smith', 0),
10 (5, 'Charlie', 'Williams', 0),
11 (6, 'Emma', 'Brown', 0),
12 (7, 'David', 'Davis', 0),
13 (8, 'Olivia', 'Miller', 0),
14 (9, 'Michael', 'Anderson', 0),
15 (10, 'Sophia', 'Taylor', 0),
16 (11, 'Robert', 'Martin', 1),
17 (12, 'Emily', 'Moore', 1),
18 (13, 'James', 'Lee', 1),
19 (14, 'Emma', 'White', 1),
20 (15, 'William', 'Johnson', 1),
21 (16, 'Isabella', 'Harris', 1),
22 (17, 'Liam', 'Smith', 1),
23 (18, 'Ava', 'Brown', 1),
24 (19, 'Ethan', 'Jones', 1),
25 (20, 'Amelia', 'Davis', 1),
26 (21, 'Mason', 'Miller', 1),
27 (22, 'Grace', 'Taylor', 1),
28 (23, 'Aiden', 'Anderson', 1),

```

```

29 (24, 'Sophie', 'Thomas', 1),
30 (25, 'Alexander', 'Moore', 1),
31 (26, 'Ella', 'Clark', 1),
32 (27, 'Logan', 'Hall', 1),
33 (28, 'Chloe', 'Young', 1),
34 (29, 'Lucas', 'Wright', 1),
35 (30, 'Lily', 'Hill', 1),
36 (31, 'Jack', 'Wilson', 1),
37 (32, 'Addison', 'King', 1),
38 (33, 'Daniel', 'Hill', 1),
39 (34, 'Natalie', 'Wilson', 1),
40 (35, 'Emily', 'King', 1),
41 (36, 'Michael', 'Clark', 1),
42 (37, 'Olivia', 'Wright', 1),
43 (38, 'Mason', 'Hall', 1),
44 (39, 'Emma', 'Young', 1),
45 (40, 'Sophia', 'Thomas', 1),
46 (41, 'Aiden', 'Moore', 1),
47 (42, 'Sophie', 'White', 1),
48 (43, 'Alexander', 'Harris', 1),
49 (44, 'Ella', 'Smith', 1),
50 (45, 'Logan', 'Brown', 1),
51 (46, 'Chloe', 'Davis', 1),
52 (47, 'Lucas', 'Taylor', 1),
53 (48, 'Lily', 'Anderson', 1),
54 (49, 'Jack', 'Martin', 1),
55 (50, 'Addison', 'Miller', 1),
56 (51, 'Daniel', 'Johnson', 1),
57 (52, 'Natalie', 'Lee', 1),
58 (53, 'Emily', 'Clark', 1),
59 (54, 'Michael', 'Young', 1),
60 (55, 'Olivia', 'Hall', 1),

61 (56, 'Mason', 'Wilson', 1),
62 (57, 'Emma', 'King', 1),
63 (58, 'Sophia', 'Wright', 1),
64 (59, 'Aiden', 'Hill', 1),
65 (60, 'Sophie', 'Thomas', 1),
66 (61, 'Alexander', 'Moore', 1),
67 (62, 'Ella', 'Clark', 1),
68 (63, 'Logan', 'Hall', 1),
69 (64, 'Chloe', 'Young', 1),
70 (65, 'Lucas', 'Wright', 1),
71 (66, 'Lily', 'Hill', 1),
72 (67, 'Jack', 'Wilson', 1),
73 (68, 'Addison', 'King', 1),
74 (69, 'Daniel', 'Hill', 1),
75 (70, 'Natalie', 'Wilson', 1),
76 (71, 'Emily', 'King', 1),
77 (72, 'Michael', 'Clark', 1),
78 (73, 'Olivia', 'Wright', 1),
79 (74, 'Mason', 'Hall', 1),
80 (75, 'Emma', 'Young', 1),
81 (76, 'Sophia', 'Thomas', 1),
82 (77, 'Aiden', 'Moore', 1),
83 (78, 'Sophie', 'White', 1),
84 (79, 'Alexander', 'Harris', 1),
85 (80, 'Ella', 'Smith', 1),
86 (81, 'Logan', 'Brown', 1),
87 (82, 'Chloe', 'Davis', 1),
88 (83, 'Lucas', 'Taylor', 1),
89 (84, 'Lily', 'Anderson', 1),
90 (85, 'Jack', 'Martin', 1),
91 (86, 'Addison', 'Miller', 1),
    (87, 'Daniel', 'Johnson', 1);

```

```

1  -- -----
2  -- Dummy data telefon
3  -- -----
4  INSERT INTO telefon(tlf, landskode, id)
5  VALUES(12345678, 47, 1),
6  (12345679, 47, 2),
7  (12345680, 47, 3),
8  (12345681, 47, 4),
9  (12345682, 47, 5),
10 (12345683, 47, 6),
11 (12345684, 47, 7),
12 (12345685, 47, 8),
13 (12345686, 47, 9),
14 (12345687, 47, 10),
15 (12345688, 47, 11),
16 (12345689, 47, 12),
17 (12345690, 47, 13),

18 (12345691, 47, 14),
19 (12345692, 47, 15),
20 (12345693, 47, 16),
21 (12345694, 47, 17),
22 (12345695, 47, 18),
23 (12345696, 47, 19),
24 (12345697, 47, 20),
25 (12345698, 47, 21),
26 (12345699, 47, 22),
27 (12345700, 47, 23),
28 (12345701, 47, 24),
29 (12345702, 47, 25),
30 (12345703, 47, 26),
31 (12345704, 47, 27),
32 (12345705, 47, 28),
33 (12345706, 47, 29),
34 (12345707, 47, 30),

```

```

35 (12345708, 47, 31),
36 (12345709, 47, 32),
37 (12345710, 47, 33),
38 (12345711, 47, 34),
39 (12345712, 47, 35),
40 (12345713, 47, 36),
41 (12345714, 47, 37),
42 (12345715, 47, 38),
43 (12345716, 47, 39),
44 (12345717, 47, 40),
45 (12345718, 47, 41),
46 (12345719, 47, 42),
47 (12345720, 47, 43),
48 (12345721, 47, 44),
49 (12345722, 47, 45),
50 (12345723, 47, 46),
51 (12345724, 47, 47),
52 (12345725, 47, 48),
53 (12345726, 47, 49),
54 (12345727, 47, 50),
55 (12345728, 47, 51),
56 (12345729, 47, 52),
57 (12345730, 47, 53),
58 (12345731, 47, 54),
59 (12345732, 47, 55),
60 (12345733, 47, 56),
61 (12345734, 47, 57),
62 (12345735, 47, 58),
63 (12345736, 47, 59),

64 (12345737, 47, 60),
65 (12345738, 47, 61),
66 (12345739, 47, 62),
67 (12345740, 47, 63),
68 (12345741, 47, 64),
69 (12345742, 47, 65),
70 (12345743, 47, 66),
71 (12345744, 47, 67),
72 (12345745, 47, 68),
73 (12345746, 47, 69),
74 (12345747, 47, 70),
75 (12345748, 47, 71),
76 (12345749, 47, 72),
77 (12345750, 47, 73),
78 (12345751, 47, 74),
79 (12345752, 47, 75),
80 (12345753, 47, 76),
81 (12345754, 47, 77),
82 (12345755, 47, 78),
83 (12345756, 47, 79),
84 (12345757, 47, 80),
85 (12345758, 47, 81),
86 (12345759, 47, 82),
87 (12345760, 47, 83),
88 (12345761, 47, 84),
89 (12345762, 47, 85),
90 (12345763, 47, 86),
91 (12345764, 47, 87);

```

```

-- -----
-- Dummy data lokasjon
-- -----

INSERT INTO lokasjon(adresse, gatenummer,
id, sted)
VALUES('Main St', 123, 1, 'CityA'),
('Oak St', 456, 2, 'CityB'),
('Maple St', 789, 3, 'CityC'),
('Elm St', 101, 4, 'CityD'),
('Cedar St', 121, 5, 'CityE'),
('Pine St', 141, 6, 'CityF'),
('Spruce St', 161, 7, 'CityG'),
('Birch St', 181, 8, 'CityH'),
('Cherry St', 201, 9, 'CityI'),
('Willow St', 221, 10, 'CityJ'),
('Pine St', 567, 11, 'CityK'),
('Cedar St', 876, 12, 'CityL'),

('Maple St', 765, 13, 'CityM'),
('Oak St', 654, 14, 'CityN'),
('Main St', 543, 15, 'CityO'),
('Spruce St', 432, 16, 'CityP'),
('Elm St', 321, 17, 'CityQ'),
('Cherry St', 210, 18, 'CityR'),
('Birch St', 109, 19, 'CityS'),
('Pine St', 876, 20, 'CityT'),
('Cedar St', 765, 21, 'CityU'),
('Maple St', 654, 22, 'CityV'),
('Oak St', 543, 23, 'CityW'),
('Main St', 432, 24, 'CityX'),
('Spruce St', 321, 25, 'CityY'),
('Elm St', 210, 26, 'CityZ'),
('Cherry St', 109, 27, 'CityAA'),
('Birch St', 987, 28, 'CityAB'),
('Pine St', 876, 29, 'CityAC'),

```

```

('Cedar St', 765, 30, 'CityAD'),
('Maple St', 654, 31, 'CityAE'),
('Oak St', 543, 32, 'CityAF'),
('Main St', 432, 33, 'CityAG'),
('Spruce St', 321, 34, 'CityAH'),
('Elm St', 210, 35, 'CityAI'),
('Cherry St', 109, 36, 'CityAJ'),
('Birch St', 987, 37, 'CityAK'),
('Pine St', 876, 38, 'CityAL'),
('Cedar St', 765, 39, 'CityAM'),
('Maple St', 654, 40, 'CityAN'),
('Oak St', 543, 41, 'CityAO'),
('Main St', 432, 42, 'CityAP'),
('Spruce St', 321, 43, 'CityAQ'),
('Elm St', 210, 44, 'CityAR'),
('Cherry St', 109, 45, 'CityAS'),
('Birch St', 987, 46, 'CityAT'),
('Pine St', 876, 47, 'CityAU'),
('Cedar St', 765, 48, 'CityAV'),
('Maple St', 654, 49, 'CityAW'),
('Oak St', 543, 50, 'CityAX'),
('Main St', 432, 51, 'CityAY'),
('Spruce St', 321, 52, 'CityAZ'),
('Elm St', 210, 53, 'CityBA'),
('Cherry St', 109, 54, 'CityBB'),
('Birch St', 987, 55, 'CityBC'),
('Pine St', 876, 56, 'CityBD'),
('Cedar St', 765, 57, 'CityBE'),
('Maple St', 654, 58, 'CityBF'),

('Oak St', 543, 59, 'CityBG'),
('Main St', 432, 60, 'CityBH'),
('Spruce St', 321, 61, 'CityBI'),
('Elm St', 210, 62, 'CityBJ'),
('Cherry St', 109, 63, 'CityBK'),
('Birch St', 987, 64, 'CityBL'),
('Pine St', 876, 65, 'CityBM'),
('Cedar St', 765, 66, 'CityBN'),
('Maple St', 654, 67, 'CityBO'),
('Oak St', 543, 68, 'CityBP'),
('Main St', 432, 69, 'CityBQ'),
('Spruce St', 321, 70, 'CityBR'),
('Elm St', 210, 71, 'CityBS'),
('Cherry St', 109, 72, 'CityBT'),
('Birch St', 987, 73, 'CityBU'),
('Pine St', 876, 74, 'CityBV'),
('Cedar St', 765, 75, 'CityBW'),
('Maple St', 654, 76, 'CityBX'),
('Oak St', 543, 77, 'CityBY'),
('Main St', 432, 78, 'CityBZ'),
('Spruce St', 321, 79, 'CityCA'),
('Elm St', 210, 80, 'CityCB'),
('Cherry St', 109, 81, 'CityCC'),
('Birch St', 987, 82, 'CityCD'),
('Pine St', 876, 83, 'CityCE'),
('Cedar St', 765, 84, 'CityCF'),
('Maple St', 654, 85, 'CityCG'),
('Oak St', 543, 86, 'CityCH'),
('Main St', 432, 87, 'CityCI');

-- -----
-- Dummy data dager
-- -----
INSERT INTO dager (dag, ukedag)
VALUES (1, 'mandag'),
(2, 'tirsdag'),
(3, 'onsdag'),
(4, 'torsdag'),
(5, 'fredag'),
(6, 'lørdag'),
(7, 'søndag');

-- -----
-- Dummy data nivaa
-- -----
INSERT INTO nivaa (n_id, beskrivelse)
VALUES (1, 'styre:3x8reps/kondisjon:10x1min, 15sek pause'),

```

```
(2, 'style:5x10reps/kondisjon:12x3min, 13sek pause'),
(3, 'style:6x12reps/kondisjon:13x4min, 10sek pause');

-- -----
-- Dummy data pulssoner
-- -----

INSERT INTO pulssoner(pulsSone, navn, beskrivelse)
VALUES(1, 'Sone 1', 'Fra 60 til 72 % av makspuls'),
(2, 'Sone 2', 'Fra 72 til 82 % av makspuls'),
(3, 'Sone 3', 'Fra 82 til 87 % av makspuls'),
(4, 'Sone 4', 'Fra 87 til 92 % av makspuls');

-- -----
-- Dummy data oktTyper
-- -----

INSERT INTO oktTyper(id, kategori)
VALUES(0, 'styrke'),
(1, 'kondisjon'),
(2, 'konkuranse');

-- -----
-- Dummy data styrkeTrening
-- -----

INSERT INTO styrkeTrening(kategori, treningstyper)
VALUES(0, 'armer'),
(0, 'ben'),
(0, 'overkropp'),
(0, 'rygg'),
(0, 'spesialisert');

-- -----
-- Dummy data styrkeOvelser
-- -----

INSERT INTO styrkeOvelser(ovelse, b_id, treningstyper)
VALUES ('biceps', 25, 'armer'),
('bryst', 24, 'overkropp'),
('chin-ups', 23, 'overkropp'),
('hofter', 22, 'rygg'),
('lår', 21, 'ben'),
('legger', 20, 'ben'),
('mage', 19, 'overkropp'),
('Markløft', 18, 'spesialisert'),
('overarm', 17, 'armer'),
('roing', 16, 'spesialisert'),
('rygghev', 15, 'rygg'),
('setemuskelen', 14, 'rygg'),
('skuldre', 13, 'rygg'),
('triceps', 12, 'armer');
```

```
('underarm', 11, 'armer');

-- -----
-- Dummy data kondisjonsOvelser
-- -----

INSERT INTO kondisjonsOverlser(kategori, ovelse, b_id)
VALUES(1, 'Alternativ kortintervall', 10),
(1, 'Hurtigkjøring i kupert terreng', 9),
(1, 'Kortintervall (høy intensitet)', 8),
(1, 'Langintervall (moderat intensitet)', 7);

-- -----
-- Dummy data konkurranse
-- -----

INSERT INTO konkurranse(kategori, event, b_id)
VALUES(2, '3000m', 5),
(2, 'Atlas stones', 4),
(2, 'Orienteringsløp', 6),
(2, 'Pulling', 3),
(2, 'Stafettløp', 2),
(2, 'Styrkeløft', 1);

-- -----
-- Dummy data oktBeskrivelse
-- -----

INSERT INTO oktBeskrivelse(b_id, beskrivelse)
VALUES (1, 'En konkurranse øvelse der man har tre forsøk på å løfte en vektstang med maksimal vekt i
øvelsene knebøy, benkpress og markløft, i den rekkefølgen som er beskrevet.'),
(2, 'Stafettløp er en konkurranse med flere lag, der deltakerne på et lag avløser hverandre og
tilbakelegger en del av den totale distansen.'),
(3, 'Deltakeren skal dra til seg en vekt eller objekt med tyngde over målstreken ved å trekke i et tau.
Tauets lengde kan variere mellom 10 til 25 meter.'),
(4, 'En konkurranse der deltakerne skal løfte vekter, med økende tyngde, opp på en plattform. '),
(5, '3000 meter løp brukes for å teste en persons utholdenhet over lengre tid. Målet er at man fra en
startlinje skal nå 3000 meter innen en hvis tidsgrense, flere runder rundt en bane eller en løype. Med
flere deltakere brukes en fellesstart.'),
(6, 'En konkurranse der en person eller flere deltakere skal besøke flere poster og returnere til
mållinjen. Første person eller lag som kommer til målstreken vinner. Man kan også gi poeng ut ifra hvor
mye tid som er brukt.'),
```

(7, '5 x 8 minutter, 2 minutters pause. Finn et kupert terreng i din idrett. Intensiteten blir moderat siden dragene er lange og pausene korte. Intervallen kan selvsagt også utføres innendørs. Du kan også kjøre et sammenhengende drag på 30-40 minutter, men dette er ofte mer mentalt krevende.1 Hurtigkjøring i kupert terreng Løp, sykle eller gå på ski i et variert terreng i cirka 60 minutter. De første 10 minuttene bør være rolige. Under økta bruker du terrenget aktivt'),

(8, '10 x 2 minutter, 2 minutters pause. Du kan legge opp intervallene på en relativt flat løype. Sett en startstrek og løp (eventuelt sykle, gå på ski eller lignende) i 2 minutter. Sett en strek der du er når det har gått to minutter. Etter pause, løp så tilbake. Under intervallene kan du kontrollere om du løper like langt per drag. Dersom du kunne løpt raskere, vet du det til nese gang. Intervallene kan selvsagt også gjøres innendørs på en tredemølle'),

(9, 'Løp, sykle eller gå på ski i et variert terreng i cirka 60 minutter. De første 10 minuttene bør være rolige. Under økta bruker du terrenget aktivt.'),

(10, '45 sekunder x 20 i motbakkeløp, 15 sekunder pause. Under denne type intervall kan du holde en høy intensitet over lang tid. Denne formen for intervalltrening er motiverende siden dragene ikke er så lange. Samtidig vil du ha en høy hjerterefrekvens siden pausene er korte.'),

(11, 'Bruk en vekt i hver hånd, bøy albue og løft vektene mot skuldrene.'),

(12, 'Bruk en vekt i begge hender, senk vekten bak hodet ved å bøye albue og strekk deretter armene.'),

(13, 'Hold en vekt i hver hånd, løft vektene rett opp fra skuldrene med utstrakte armer.'),

(14, 'Ligg på ryggen, bøy knærne og løft hoftene opp mot taket.'),

(15, 'Ligg på magen, løft overkroppen og bena opp fra bakken.'),

(16, 'Bøy deg fremover, hold vektene i hendene, og dra dem opp mot magen.'),

(17, 'Hold en vekt i hver hånd, løft vektene ut til sidene med utstrakte armer.'),

(18, 'Stå med rett rygg, bøy deg fremover og løft vektene fra gulvet ved å rette ut hoftene.'),

(19, 'Ligg på ryggen, løft overkroppen opp mot knærne.'),

(20, 'Stå oppreist, løft hælne slik at du står på tærne, senk hælne ned igjen.'),

(21, 'Plasser en vekt på skuldrene, bøy knærne og senk kroppen nedover som om du setter deg på en stol.'),

(22, 'Lig på ryggen, bøy knærne, og sving hoftene opp og ned.'),

(23, 'Heng på en stang med overhåndsgrep, løft kroppen opp slik at haken passerer stangen.'),

(24, 'Ligg på ryggen, senk vektene ned til brystet, og press dem opp igjen.'),

(25, 'Hold en vekt i hver hånd, senk armene og bøy albue for å løfte vektene mot skuldrene.');

```
-- -----
-- Dummy data treningsplan
-- -----

INSERT INTO treningsplan (lopenummer, t_id, p_id, dag, okt_t, muskelgruppe, ovelse, nivaa)
VALUES
(1, 1, 1, 1, 0, 'armer', 'biceps', 1),
(2, 1, 1, 1, 0, 'armer', 'overarm', 1),
(3, 1, 1, 1, 0, 'armer', 'triceps', 1),
(4, 1, 1, 1, 0, 'armer', 'underarm', 1),
(5, 1, 1, 1, 0, 'armer', 'underarm', 1),
(6, 1, 1, 2, 0, 'ben', 'lår', 2),
(7, 1, 1, 2, 0, 'ben', 'legger', 2),
(8, 1, 1, 2, 0, 'ben', 'setemuskel', 2),
(9, 1, 1, 3, 0, 'overkropp', 'bryst', 1),
(10, 1, 1, 3, 0, 'overkropp', 'mage', 1),
(11, 1, 1, 4, 0, 'rygg', 'rugghev', 1),
(12, 1, 1, 4, 0, 'rygg', 'skuldre', 1),
```

```

(13, 1, 1, 5, 0, 'spesialisert', 'markløft', 2),
(14, 1, 1, 5, 0, 'spesialisert', 'roing', 3),
(15, 2, 5, 1, 1, NULL, 'Alternativ kortintervall', 1),
(16, 2, 5, 2, 1, NULL, 'Hurtigkjøring i kupert terreng', 2),
(17, 2, 5, 3, 1, NULL, 'Kortintervall (høy intensitet)', 3),
(18, 2, 5, 4, 1, NULL, 'Langintervall (moderat intensitet)', 3),
(19, 3, 7, 1, 0, 'overkropp', 'bryst', 2),
(20, 3, 7, 1, 0, 'spesialisert', 'roing', 1),
(21, 3, 7, 1, 0, 'rygg', 'skuldre', 1),
(22, 3, 7, 1, 0, 'armer', 'biceps', 2),
(23, 3, 7, 1, 0, 'armer', 'triceps', 2),
(24, 3, 7, 2, 1, 'armer', 'triceps', 2),
(25, 3, 7, 4, 0, 'ben', 'lår', 2),
(26, 3, 7, 4, 0, 'ben', 'setemuskelen', 2),
(27, 3, 7, 4, 0, 'ben', 'legger', 3),
(28, 3, 7, 5, 0, 'ben', 'legger', 3),
(29, 3, 7, 2, 0, NULL, 'Kortintervall (høy intensitet)', 2),
(30, 3, 7, 6, 2, NULL, 'Orienteringsløp', 3);

-- -----
-- Dummy data okt
-- -----

INSERT INTO okt(lopenummer, p_id, o_id, dato, varighet_min, vektor_kg, pulsSone, plassering,
kommentar,  ovelse, event, treningsplan)
VALUES(2, 56, 0, '2024-01-15', 20, 25, 2, NULL, 'God start på nyttårsforsett', 'overarm',
NULL, 1),
(3, 56, 0, '2024-01-15', 15, 20, 2, NULL, 'Litt nedfall fra fjordåret', 'triceps', NULL, 1),
(4, 56, 0, '2024-01-15', 10, 10, 1, NULL, 'Litt nedfall fra fjordåret', 'overarm', NULL, 1),
(5, 56, 0, '2024-01-15', 15, 60, 3, NULL, 'Stiv rygg, litt roligere neste gang', 'markløft',
NULL, 1),
(8, 67, 1, '2024-02-12', 30, NULL, 2, NULL, 'Kan klare et høyere nivå neste gang',
'Kortintervall (høy intensitet)', NULL, 3),
(11, 67, 2, '2024-02-15', 30, NULL, 3, 2, 'Det var nesten første plass!', NULL,
'Orienteringsløp', 3),
(12, 12, 2, '2024-02-15', 30, NULL, 3, 1, 'første plass!', NULL, 'Orienteringsløp', 3);

```

SQL-spørringer

Her foreligger det et tilnærmet ubegrenset antall med mulige løsninger som kan settes opp etter klientens ønsker. Vi har i denne oppgaven valgt ut et sett med eksempel spørringer som vi tenker oss kunne vært aktuelt i en prospekt plan. Her har vi fokus på spørringer som faller innenfor oppgavens kravspesifikasjoner. Alle spørringer i tabellen under viser til ‘views’ som er opprettet i databasen, med en kort beskrivelse av spørringens funksjon, samt selve SQL spørringen.

View navn	Beskrivelse	SQL spørring
-----------	-------------	--------------

view_tranier (ment for trener)	Denne henter ut en liste over trenere fra person tabellen	<pre> SELECT p.fornavn AS fornavn, p.etternavn AS etternavn, t.tlf AS tlf, CONCAT(l.adresse, ' ', l.gatenummer) AS adresse, CONCAT(s.postnummer, ' ', l.sted) AS postnummer FROM person p JOIN bedriftsforhold b ON p.status = b.status JOIN telefon t ON p.id = t.id JOIN lokasjon l ON p.id = l.id JOIN stedsnavn s ON l.sted = s.sted WHERE b.status = 0 </pre>
view_utover (ment for trener)	Denne henter ut en liste over utøvere fra person tabellen	<pre> SELECT p.fornavn AS fornavn, p.etternavn AS etternavn, t.tlf AS tlf, CONCAT(l.adresse, ' ', l.gatenummer) AS adresse, CONCAT(s.postnummer, ' ', l.sted) AS postnummer FROM person p JOIN bedriftsforhold b ON p.status = b.status JOIN telefon t ON p.id = t.id JOIN lokasjon l ON p.id = l.id JOIN stedsnavn s ON l.sted = s.sted WHERE b.status = 1 </pre>
View_konkuranser (ment for trener)	Denne spørringen henter ut en oversikt over alle utførte konkurranser fra okt tabellen og sorterer de etter dato	<pre> CREATE VIEW view_konkuranser AS SELECT p_id, concat(p.fornavn, ' ', p.etternavn) AS utøver, dato, pulsSone, plassering, event, kommentar FROM okt o JOIN person p ON o.p_id = p.id WHERE o_id = 2 ORDER BY dato; </pre>
Alle_okter (ment for trener)	Denne spørringen henter ut alle tabeller uavhengig av hvilken type trening eller	<pre> SELECT p.fornavn, p.etternavn, o.* FROM okt o JOIN person p ON o.p_id = p.id ORDER BY lopenummer; </pre>

	treningsplan. Fornavn og etternavn på utøver blir også hentet fra person tabellen og lagt til i utskriften.	
Styrke_okter (ment for trener)	Denne spørringen henter ut alle utførte økter som hører til styrketrenings økter. Fornavn og etternavn på utøver blir hentet ut fra person tabellen.	<pre> SELECT o.p_id, p.fornavn, p.etternavn, dato, varighet_min, vekter_kg, pulsSone, ovelse, treningsplan FROM okt o JOIN person p ON o.p_id = p.id WHERE o_id = 0 ORDER BY lopenummer; </pre>
kondisjons_okter (ment for trener)	Denne spørringen henter ut alle utførte økter som hører til kondisjons økter. Fornavn og etternavn på utøver blir hentet ut fra person tabellen.	<pre> SELECT o.p_id, p.fornavn, p.etternavn, dato, varighet_min, pulsSone, ovelse, treningsplan FROM okt o JOIN person p ON o.p_id = p.id WHERE o_id = 1 ORDER BY lopenummer; </pre>
Konkurranse_okter (ment for trener)	Denne spørringen henter ut alle utførte økter som er relatert til konkurranser. Her kommer det også med utøvers plassering i konkurransen. Navn på utøver hentes fra person tabellen.	<pre> SELECT o.p_id, p.fornavn, p.etternavn, dato, plassering, varighet_min, pulsSone, event, treningsplan FROM okt o JOIN person p ON o.p_id = p.id WHERE o_id = 2 ORDER BY lopenummer; </pre>
styrke_beskrivelse (ment for utøver)	Denne spørringen henter ut en liste over tilgjengelige styrke øvelser, samt en beskrivelse av hvordan utøvelsen gjøres.	<pre> SELECT treningstyper, ovelse, beskrivelse FROM styrkeOvelser so JOIN oktBeskrivelse ob ON so.b_id = ob.b_id ORDER BY treningstyper; </pre>

kondisjon_beskrivelse (ment for utøver)	Denne spørringen henter ut en liste over tilgjengelige kondisjons øvelser, samt en beskrivelse av hvordan utøvelsen gjøres.	<pre>SELECT ovelse, beskrivelse FROM kondisjonsOverlser ko JOIN oktBeskrivelse ob ON ko.b_id = ob.b_id ORDER BY ovelse;</pre>
konkurranse_beskrivelse (ment for utøver)	Denne spørringen henter ut en liste over tilgjengelige konkurranser, samt en beskrivelse av hvordan disse utføres.	<pre>SELECT event, beskrivelse FROM konkurranse k JOIN oktBeskrivelse ob ON k.b_id = ob.b_id ORDER BY event;</pre>
utover_okter (ment for utøver)	Denne spørringen henter ut alle utførte økter på en spesifikk utøver. Her vil det forekomme flere NULL verdier, da forskjellige økter ikke bruker lik data. For å unngå dette slik som DB modellen vår er strukturert, må man lage en egen spørring for de forskjellige økt typene som finnes. Fra et front-end perspektiv ville man satt en variabel som knytter bruker sitt id numnmer til WHERE filtreringen i denne spørringen, slik at riktig utøver får tilgang til sine egne økter.	<pre>SELECT concat(p.fornavn, ' ', p.etternavn) AS utøver, p.id, dato, ot.kategori, ovelse, varighet_min, vektor_kg, pulsSone, event, plassering, treningsplan, kommentar FROM okt o JOIN person p ON o.p_id = p.id JOIN oktTyper ot ON o.o_id = ot.id WHERE p_id = 67;</pre>
Alle_treningsplaner (ment for trener)	Henter ut en liste over alle treningsplaner som er opprettet og sorterer de etter treningsplan id og rekkefølge på ukedager.	<pre>SELECT t.t_id AS treningsplan, d.ukedag, ot.kategori, t.muskelgruppe, t.ovelser, t.nivaa FROM treningsplan t LEFT JOIN dager d ON t.dag = d.dag LEFT JOIN oktTyper ot ON t.okt_t = ot.id ORDER BY t.t_id, d.dag;</pre>
spesifikk_treningsplan (ment for utøver)	Henter ut en liste av en spesifikk treningsplan. Her ville det vært satt	<pre>SELECT t.t_id AS treningsplan, d.ukedag, ot.kategori,</pre>

	opp en variabel løsning som endrer verdi på treningsplan id, slik at utøver hadde fått ut den treningsplanen de var satt opp mot.	<pre> t.muskelgruppe, t.ovelse, t.nivaa FROM treningsplan t LEFT JOIN dager d ON t.dag = d.dag LEFT JOIN oktTyper ot ON t.okt_t = ot.id WHERE t.t_id = 1 ORDER BY d.dag; </pre>
--	---	---

Skjermdump fra view spørringer

I denne seksjonen har vi hentet ut et skjermdump på alle overnevnte 'views' ved å bruke 'SELECT * FROM et_gitt_view' for å illustrere hvordan listene ser ut når vi kjører disse spørringene.

view_tranier

	fornavn	etternavn	tlf	adresse	postnummer
▶	John	Doe	12345678	Main St 123	1234 CityA
	Jane	Doe	12345679	Oak St 456	5678 CityB
	Alice	Johnson	12345680	Maple St 789	9101 CityC
	Bob	Smith	12345681	Elm St 101	1121 CityD
	Charlie	Williams	12345682	Cedar St 121	3141 CityE
	Emma	Brown	12345683	Pine St 141	5161 CityF
	David	Davis	12345684	Spruce St 161	7181 CityG
	Olivia	Miller	12345685	Birch St 181	9201 CityH
	Michael	Anderson	12345686	Cherry St 201	1221 CityI
	Sophia	Taylor	12345687	Willow St 221	3241 CityJ

view_utover

	fornavn	etternavn	tlf	adresse	postnummer
▶	Robert	Martin	12345688	Pine St 567	7890 CityK
	Emily	Moore	12345689	Cedar St 876	5432 CityL
	James	Lee	12345690	Maple St 765	2109 CityM
	Emma	White	12345691	Oak St 654	1098 CityN
	William	Johnson	12345692	Main St 543	987 CityO
	Isabella	Harris	12345693	Spruce St 432	9876 CityP
	Liam	Smith	12345694	Elm St 321	8765 CityQ
	Ava	Brown	12345695	Cherry St 210	7654 CityR
	Ethan	Jones	12345696	Birch St 109	6543 CityS
	Amelia	Davis	12345697	Pine St 876	5432 CityT
	Mason	Miller	12345698	Cedar St 765	4321 CityU

View_konkuranser

	p_id	utøver	dato	pulsSone	plassering	event	kommentar
▶	67	Jack Wilson	2024-02-15	3	2	Orienteringsløp	Det var nesten første plass!
	12	Emily Moore	2024-02-15	3	1	Orienteringsløp	første plass!

Alle_okter

	fornavn	etternavn	lopernummer	p_id	o_id	dato	vaighet_min	vekt_kg	pulsSone	plassering	kommentar	ovelse	event	treningsplan
▶	Mason	Wilson	2	56	0	2024-01-15	20	25	2		God start på nyårsforsett	overarm		1
	Mason	Wilson	3	56	0	2024-01-15	15	20	2		Litt nedfall fra fjordløpet	tracé		1
	Mason	Wilson	4	56	0	2024-01-15	10	10	1		Litt nedfall fra fjordløpet	overarm		1
	Mason	Wilson	5	56	0	2024-01-15	15	60	3		Stiv rygg, litt roligere neste gang	markløft		1
	Jack	Wilson	8	67	1	2024-02-12	30		2		Kan klare et høyere nivå neste gang	Kortintervall (høy intensitet)		3
	Jack	Wilson	11	67	2	2024-02-15	30		3	2	Det var nesten første plass!		Orienteringsløp	3
	Emily	Moore	12	12	2	2024-02-15	30		3	1	første plass!		Orienteringsløp	3

Styrke_okter													
		p_id	fornavn	etternavn	dato	varighet_min	vekt_kg	pulsSone	ovelse	treningsplan			
	▶	56	Mason	Wilson	2024-01-15	20	25	2	overarm	1			
		56	Mason	Wilson	2024-01-15	15	20	2	triceps	1			
		56	Mason	Wilson	2024-01-15	10	10	1	overarm	1			
		56	Mason	Wilson	2024-01-15	15	60	3	markløft	1			
kondisjons_okter													
		p_id	fornavn	etternavn	dato	varighet_min	pulsSone	ovelse	treningsplan				
	▶	67	Jack	Wilson	2024-02-12	30	2	Kortintervall (høy intensitet)		3			
Konkurranse_okter													
		p_id	fornavn	etternavn	dato	plassering	varighet_min	pulsSone	event	treningsplan			
	▶	67	Jack	Wilson	2024-02-15	2	30	3	Orienteringsløp	3			
		12	Emily	Moore	2024-02-15	1	30	3	Orienteringsløp	3			
styrke_beskrivelse													
		treningstyper		ovelse	beskrivelse								
	▶	armer	biceps	Hold en vekt i hver hånd, senk armene og bøy a...									
		armer	triceps	Bruk en vekt i begge hender, senk vekten bak h...									
		armer	overarm	Hold en vekt i hver hånd, løft vektene ut til side...									
		armer	underarm	Bruk en vekt i hver hånd, bøy albuen og løft v...									
		ben	lår	Plasser en vekt på skuldrene, bøy knærne og s...									
		ben	legger	Stå oppreist, løft hælene slik at du står på tærn...									
		overkropp	mage	Ligg på ryggen, løft overkroppen opp mot knær...									
		overkropp	chin-ups	Heng på en stang med overhåndsgrep, løft kro...									
		overkropp	bryst	Ligg på ryggen, senk vektene ned til brystet, o...									
		rygg	hofter	Lig på ryggen, bøy knærne, og sving hoftene o...									
		rygg	rygghev	Ligg på magen, løft overkroppen og bena opp fr...									
		rygg	setemus...	Ligg på ryggen, bøy knærne og løft hoftene op...									
		rygg	skuldre	Hold en vekt i hver hånd, løft vektene rett opp ...									
	spesialisert	roing	Bøy deg fremover, hold vektene i hendene, og ...										
	spesialisert	Markløft	Stå med rett rygg, bøy deg fremover og løft ve...										
kondisjon_beskrivelse													
		ovelse		beskrivelse									
	▶	Alternativ kortintervall		45 sekunder x 20 i motbakkeløp, 15 sekunder p...									
		Hurtigkjøring i kupert terreng		Løp, sykle eller gå på ski i et variert terreng i cir...									
		Kortintervall (høy intensitet)		10 x 2 minutter, 2 minutters pause. Du kan legg...									
	Langintervall (moderat intensitet)		5 x 8 minutter, 2 minutters pause. Finn et kuper...										
konkurranse_beskrivelse													
		event		beskrivelse									
	▶	3000m		3000 meter løp brukes for å teste en persons ut...									
		Atlas stones		En konkurranse der deltakerne skal løfte vekter...									
		Orienteringsløp		En konkurranse der en person eller flere deltake...									
		Pulling		Deltakeren skal dra til seg en vekt eller objekt m...									
		Stafettløp		Stafettløp er en konkurranse med flere lag, der ...									
	Styrkeløft		En konkurranse øvelse der man har tre forsøk p...										
utover_okter													
		utøver	id	dato	kategori	ovelse	varighet_min	vekt_kg	pulsSone	event	plassering	treningsplan	kommentar
	▶	Jack Wilson	67	2024-02-12	kondisjon	Kortintervall (høy intensitet)	30	0000	2	0000	0000	3	
	Jack Wilson	67	2024-02-15	konkurranse	0000	30	0000	3	Orienteringsløp	2	3		Det var nesten første plass!

Alle_treningsplaner		treningsplan	ukedag	kategori	muskelgruppe	ovelse	nivaa	
	▶	1	mandag	styrke	armer	overarm	1	
		1	mandag	styrke	armer	biceps	1	
		1	mandag	styrke	armer	underarm	1	
		1	mandag	styrke	armer	underarm	1	
		1	mandag	styrke	armer	triceps	1	
		1	tirsdag	styrke	ben	setemuskelen	2	
		1	tirsdag	styrke	ben	legger	2	
		1	tirsdag	styrke	ben	lår	2	
		1	onsdag	styrke	overkropp	mage	1	
		1	onsdag	styrke	overkropp	bryst	1	
		1	torsdag	styrke	rygg	skuldre	1	
		1	torsdag	styrke	rygg	rugghev	1	
		1	fredag	styrke	spesialisert	markløft	2	
		1	fredag	styrke	spesialisert	roing	3	
		2	mandag	kondisjon	NULL	Alternativ ko...	1	
		2	tirsdag	kondisjon	NULL	Hurtigkjøring...	2	
		2	onsdag	kondisjon	NULL	Kortintervall ...	3	
	spesifikk_treningsplan		treningsplan	ukedag	kategori	muskelgruppe	ovelse	nivaa
		▶	1	mandag	styrke	armer	underarm	1
		1	mandag	styrke	armer	underarm	1	
		1	mandag	styrke	armer	triceps	1	
		1	mandag	styrke	armer	overarm	1	
		1	mandag	styrke	armer	biceps	1	
		1	tirsdag	styrke	ben	setemuskelen	2	
		1	tirsdag	styrke	ben	legger	2	
		1	tirsdag	styrke	ben	lår	2	
		1	onsdag	styrke	overkropp	mage	1	
		1	onsdag	styrke	overkropp	bryst	1	
		1	torsdag	styrke	rygg	skuldre	1	
		1	torsdag	styrke	rygg	rugghev	1	
		1	fredag	styrke	spesialisert	roing	3	
		1	fredag	styrke	spesialisert	markløft	2	

Konklusjon

Når vi startet på prosjektet var vi noe usikre på hvilken database struktur som var mest hensiktsmessig i forhold til oppgavens kravspesifikasjon. Mye av grunnen til dette var blant annet at vi ikke hadde en reel klient og forholde oss til, slik at vi kunne fulgt opp med spørsmål om hvordan systemet skulle brukes, og om det forelå noe spesifikke ønsker om struktur. Dette førte til at vi brukte mye av startperioden på 'brainstorm' sesjoner hvor vi tenkte oss frem til hvilke data som ville være relevant for det gitte system. Etter at vi hadde hva vi anså som tilstrekkelige med attributter i en samlet entitet, startet vi å dele dette opp attributtene i entiteter gjennom normaliseringsprosessen. Når denne prosessen var gjennomført, tegnet vi opp et ER diagram som vi videre brukte til å opprette databasen og importerte dummydata til sine respektive entiteter. Det var først når vi startet med spørringer i databasen at vi innså enkelte forbedringspotensialer, samt mulige avvik i data som kunne oppstå ved senere data import. Vi konkluderte med at den seksjonen som håndterer person data hadde en tilfredsstillende struktur og burde fungere som forventet. Derimot var det litt mangler på fleksibilitet vedrørende 'økt', 'treningsplan' og 'øktTyper' entitetene. Dette er punkter som vi ville ha fokusert på ved en videre utvikling av dette systemet:

- Slik systemet er satt opp i dag må beskrivelser på de forskjellige økt typene hentes ut separat (en liste for styrketrenig, en for kondisjon og en for konkurranser). En bedre struktur på nøkkelverdier ville ført til at vi kunne ha hentet ut disse som en samlet liste om ønskelig.
- Det foreligger ingen direkte kobling mellom utøvers økt og treningsplan. Selv om det finnes en attributt i 'økt' entiteten som samsvarer med en id verdi på 'treningsplaner' entiteten, så foreligger det ikke noen direkte kobling mellom disse i denne database strukturen. Dette ville også kunne la seg løse ved å endre på nøkkelverdier, slik som å gjøre om PK i 'økt' entiteten til en sammensatt nøkkel og opprette en ny FK mellom treningsplan id på disse to entitetene.

Mot slutten av dette prosjektet fikk vi med Ivar som en ekstra deltager. Han satt seg raskt inn i den strukturen vi allerede hadde etablert og har videre bistått med mye god innsikt om hvordan løsninger kan forbedres og totalt sett blitt ansett som en svært god resurs i dette prosjektet.

I dette eksamensprosjektet har vi gjort oss mange gode erfaringer, samt sett løsninger som vi kunne ha løst på en bedre måte, om dette systemet skulle ha blitt vedlikeholdt med videre utvikling/oppdatering.

Litteraturliste

SATS. (2023) KONDISJONSTRENING - DEN ULTIMATE GUIDEN

<https://www.sats.no/magasin/trening/kondisjonstrening/kondisjonstrening-den-ultimate-guiden>

Holmemo, J. (2019, 6. oktober). Sett sammen din egen styrkeøkt!

<https://www.trening.no/styrke/sett-sammen-din-egen-styrkeokt/>

Holmemo, J. (2019, 30. juli). Innføring i pulssone. <https://www.trening.no/utholdenhet/innforing-i-pulssoner/>

OpenAI. (2023). ChatGPT (versjon 3.5) [Stor språkmodell].

<https://chat.openai.com/share/95369439-b08f-4774-864b-2de4a5e4d28b>